Hazard Analysis
LingoLyrics

Team #21, LingoLyrics
Rhea Gokhale
Aansh Kotian
Omar Bakr
Raymond Ma

Table 1: Revision History

Date	Developer(s)	Change
October 18, 2026	Rhea Gokhale	Initial Revision 0 draft
October 18, 2026	Rhea Gokhale	Addressed review comments: reviewed all ratings against the scales, added HS8 and HS9, narrowed HS3, IN5, IN7, and AU1 to the capstone scope, added the residual risk table, the verification of content generated by AI, and references
October 19, 2026	Rhea Gokhale	Revised against the grading rubric: added the document overview, lists of tables and figures, the system context figure and interface table, the description of how the FMEA was carried out, assumption CA3, failure modes H4.5 and H7.3 with requirement IM3 and the extended IM2, the completed reflection, and tool versions in the AI disclosure

Contents

1 Introduction **1**

2 Scope and Purpose of Hazard Analysis **1**

3 System Boundaries and Components **2**

4 Critical Assumptions **3**

5 Failure Mode and Effect Analysis **4**
- 5.1 How the Analysis Was Carried Out . . . 4
- 5.2 Hazards Out of Scope . . . 5
- 5.3 Rating Scales . . . 5
- 5.4 FMEA Table . . . 5

6 Safety and Security Requirements **14**
- 6.1 Health and Safety Requirements . . . 14
- 6.2 Access Requirements . . . 15
- 6.3 Integrity Requirements . . . 15
- 6.4 Privacy Requirements . . . 16
- 6.5 Audit Requirements . . . 16
- 6.6 Immunity Requirements . . . 16
- 6.7 Legal Requirements . . . 16

7 Roadmap **16**
- 7.1 Residual Risk . . . 17

References **18**

8 Appendix: Generative AI Use Disclosure **20**
- 8.1 AI Tools Used . . . 20
- 8.2 How and Where Used . . . 20
- 8.3 Verification of Content Generated by AI . . . 20
- 8.4 Limitations . . . 21

9 Appendix: Reflection **22**

List of Tables

1 Revision History i
2 Inputs and outputs that cross the system boundary 4
3 Rating scales 5
4 FMEA Worksheet 6
5 High priority failure modes, in order of RPN 14
6 Residual risk accepted for each high priority failure mode 17
7 AI tools used for this document 20
8 Where AI was used in this document 20

List of Figures

1 System context of LingoLyrics for the hazard analysis 2

1 Introduction

This document is the hazard analysis for LingoLyrics, a language learning product based on songs. It is made up of a mobile learner application and a song map editor that runs in a web browser. Learners study synchronized, annotated lyrics on their phone and send selected vocabulary to Anki. Creators use the editor to prepare the song maps that drive each lesson.

LingoLyrics is not a system on which safety critically depends: it controls no machinery and cannot directly injure anyone. It is, however, used by the general public, often with headphones and on the move, and it writes into the learner's own Anki collection. Its hazards are therefore mostly about hearing and attention, incorrect learning content, loss or corruption of user data, privacy, legal exposure from song content, and the learner's wellbeing.

Definition of Hazard

Following Nancy Leveson's work [1], a **hazard** is a property or condition of LingoLyrics that, together with a condition in its environment, can lead to a **loss**. A **loss** is any outcome that a stakeholder would find unacceptable. The losses considered in this document are listed in Section 2.

Intended Audience

This document is written for the LingoLyrics team, who will implement the requirements it produces, for the teaching staff and peer reviewers who assess it, and for anyone who later extends the product and needs to know which risks have already been considered.

Changes to the Template

The template has been followed with five additions. Section 3 adds a system context figure and a table of the inputs and outputs that cross the system boundary. Section 5 explains how the analysis was carried out and adds a *Hazards Out of Scope* subsection. The FMEA table rates each failure mode for severity, occurrence, and detection and gives its risk priority number (RPN). The RPN is used to prioritize the requirements in the Roadmap (Section 7), which also records the residual risk accepted for each high priority failure mode. A list of references is given at the end of the document.

Document Overview

The rest of the document is organized as follows.

- Section 2 states the purpose and scope of the analysis and the losses it is concerned with.
- Section 3 defines the system boundary, its components, and what crosses the boundary.
- Section 4 lists the critical assumptions.
- Section 5 describes the method, the rating scales, and the FMEA table itself.
- Section 6 lists the safety and security requirements that the FMEA produced.
- Section 7 says when each requirement will be implemented and which risks remain afterwards.
- The appendices give the disclosure of the use of generative AI and the team's reflection.

2 Scope and Purpose of Hazard Analysis

The purpose of this analysis is to identify the ways the components of LingoLyrics can fail or be misused, the effects and causes of those failures, and the requirements or other mitigations that reduce each risk to an acceptable level. Requirements discovered here are listed in Section 6 and are included in the Software Requirements Specification (SRS).

The analysis covers both failures (the system does something wrong) and hazards that exist when the system works as designed (for example, a correctly synchronized lesson that distracts a learner who is walking in traffic).

The losses that the hazards in this document could lead to are:

- **L1 Physical harm:** hearing discomfort or damage from unexpected audio, injury from distraction while moving, or discomfort for photosensitive users from flashing highlights.
- **L2 Incorrect learning:** the learner learns a wrong meaning, reading, pronunciation, or word boundary, or unknowingly learns an offensive term, and carries that error into real conversations and into their Anki reviews.
- **L3 Loss or corruption of user data:** notes in the learner's Anki collection are duplicated, malformed, or overwritten, selected vocabulary is silently lost, or a creator loses timing and annotation work.
- **L4 Privacy and security:** personal data (including the device's IP address) leaves the device without consent, or a malicious song map or misconfigured integration allows an attacker to run code or change the learner's data.
- **L5 Legal:** distributing lyrics or recordings without the right to do so.
- **L6 Wellbeing:** the product causes stress, discouragement, or compulsive use, for example through manipulative engagement mechanics.

3 System Boundaries and Components

Figure 1 shows the system boundary. Everything inside the dashed box is built by the team and is analyzed component by component. Everything outside it is part of the environment that the system depends on or affects.

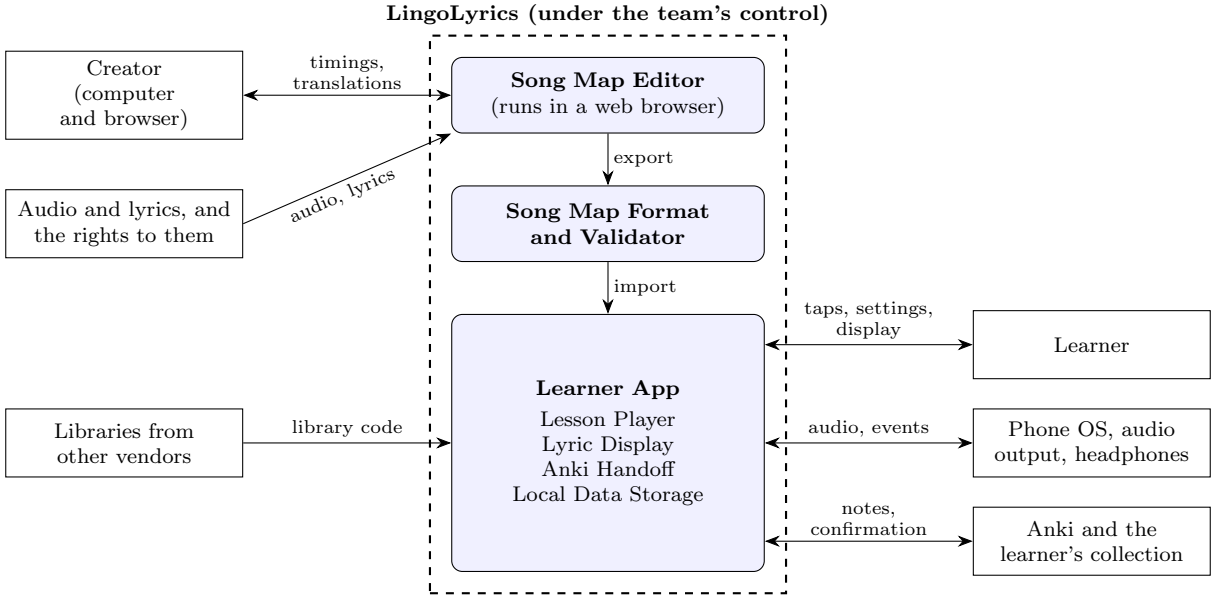


Figure 1: System context of LingoLyrics for the hazard analysis

The system under analysis consists of the following components, which are under the team's control:

1. **Lesson Player** (learner app): plays the song's audio and highlights the current lyric line and word using the song map's timestamps.

2. **Lyric Display and Language Support** (learner app): renders lyrics in their original script, the stored word segmentation, and the stored transliteration.
3. **Annotation and Lesson Content**: translations, contextual annotations, and content metadata contained in each song map and shown to the learner.
4. **Song Map Format and Validator**: the shared JSON Schema and the validation code used by both the editor (on export) and the learner app (on import).
5. **Song Map Editor** (web): the creator's tool for aligning timing, entering translations and annotations, verifying segmentation and transliteration, and exporting song maps.
6. **Anki Handoff**: the learner app's module that sends a selected vocabulary item to Anki through the AnkiDroid API [6], AnkiConnect [7], or an export file (the method is decided by the Proof of Concept, risk R3 in the Development Plan).
7. **Local Data Storage** (learner app): stored lessons, display preferences, and the queue of vocabulary waiting to be sent to Anki.

The following elements are outside the team's control but are still critical to the system, so they are included as sources of failure where relevant:

- the learner's phone and operating system (iOS or Android), including system volume, audio routing, and headphones,
- the creator's computer and web browser,
- Anki, AnkiDroid, and the AnkiConnect extension, and the learner's Anki collection,
- the audio and lyric content itself and the rights attached to it,
- the libraries from other vendors that the app and the editor are built with.

Table 2 lists what crosses the boundary in each direction and the FMEA rows (Section 5) that deal with each one. Every input and output was checked for at least one failure mode, so that no interface was left out of the analysis.

The initial release has no user accounts, no backend server, and no cloud sync, as stated in the Problem Statement and Development Plan. Hazards specific to those features are therefore not analyzed. They must be added if those features are introduced.

4 Critical Assumptions

The team has kept assumptions to a minimum. In particular, song maps are *not* assumed to be free of errors, song map files are *not* assumed to come from a trusted source, and learners are *not* assumed to use the app only while stationary. All three are treated as hazards in Section 5.

CA1: Software from other vendors (the phone operating system, the web browser, Anki, AnkiDroid, and AnkiConnect) behaves as described in its public documentation. Failures caused by defects inside that software are out of scope, although the app must still detect and report a failed handoff (H6.4).

CA2: The learner's phone meets the minimum operating system version supported by the Expo SDK release the app is built with.

CA3: In the initial release, the team members who create and review lessons act in good faith. They may make mistakes, which are treated as hazards (H3.1, H3.2, H5.2, H5.4), but they do not deliberately enter false reviewer names or licence information. This matters because IN5 and LG1 rely on names and licences entered by people and cannot verify them without user accounts. Song maps from anyone outside the team are still treated as untrusted (H4.3 to H4.5).

Each assumption will be checked again when the scope changes. CA3 in particular no longer holds once creators outside the team can publish lessons, and the hazards it covers would then need verified identities.

Table 2: Inputs and outputs that cross the system boundary

Dir.	Item	From or to	FMEA rows
In	Audio file and lyrics	Creator, content owner	H1.2, H1.3, H2.3, H5.4
In	Timings, translations, annotations, verification	Creator	H3.1, H3.2, H5.1 to H5.3
In	Song map package	File imported by the learner	H4.1 to H4.5
In	Play, pause, seek, select, and send actions	Learner	H1.2, H6.1
In	Audio route changes and app lifecycle events	Phone OS	H1.1, H1.2, H7.1
In	Text size and reduce motion settings	Phone OS	H2.4, H2.5
In	Responses to note requests	Anki	H6.4
In	Library code	Other vendors	H7.2, H7.3
Out	Audio playback	Headphones or speaker	H1.2
Out	Lyrics, highlighting, and annotations	Screen	H1.1, H1.4, H2.1 to H2.5, H3.1 to H3.3
Out	New notes	Anki collection	H6.1 to H6.3, H6.5
Out	Exported song map	File	H4.1 to H4.5, H5.2
Out	Error reports and exported queue	Share sheet, file	H3.1, H6.4, H7.2
Out	Permission requests	Phone OS	H6.6

5 Failure Mode and Effect Analysis

A Failure Modes and Effects Analysis (FMEA) was used to identify and analyze hazards component by component. Each row lists a failure mode, its effects (with the related loss from Section 2), its causes, how the failure is detected, the recommended action, its ratings for severity, occurrence, and detection with the resulting risk priority number, and the safety or security requirements (Section 6) that mitigate it. Each row has a reference number (*Ref.*) so that requirements can trace back to it.

5.1 How the Analysis Was Carried Out

FMEA was chosen because LingoLyrics is made of a small number of components with clear responsibilities (Section 3), and the requirements are still being written. Working through one component at a time makes it easy to see which component a requirement belongs to. A top down method such as fault tree analysis suits a system with one or two catastrophic top events, which LingoLyrics does not have. Its losses (L1 to L6) are many and mostly independent, so a bottom up method finds more of them.

The analysis followed these steps.

1. For each component, the team listed what it is responsible for and asked the following questions about each responsibility: What if it is not done? What if it is done wrongly, too early, or too late? What if it is done correctly but still causes harm? How could someone misuse it on purpose?

2. Each answer that could lead to a loss in Section 2 was recorded as a failure mode, with its effects and causes.

3. Each input and output in Table 2 was checked to make sure it appears in at least one failure mode.

4. The course checklist's prompt to think broadly about safety was used to look for security, privacy, legal, and wellbeing hazards that the first pass had missed. Rows such as H1.4, H2.4, H3.2, H3.3, and H6.5 come from this step.

5. Each failure mode was rated using Table 3, a recommended action was written for it, and that action was turned into one or more requirements that can be tested.

6. Every requirement was checked to trace to at least one failure mode, and every failure mode to at least one requirement.

5.2 Hazards Out of Scope

- Hardware failure of the learner's phone, headphones, or the creator's computer.
- Defects inside Anki, AnkiDroid, AnkiConnect, the operating system, or the browser (see CA1).
- The overall volume the learner chooses with their device's controls. The app controls when playback starts and stops, how it starts, and reports the loudness of each track to its creator (H1.2). Limits on listening over long periods are left to the operating system's hearing protection features.
- Hazards of streaming service integration, a production music catalogue, user accounts, and cloud sync, none of which are in the initial release.
- A user deliberately modifying the installed app or the files on their own device.

5.3 Rating Scales

Each failure mode is rated on three scales from 1 to 10. The ratings describe the failure mode *before* the recommended action is taken, so they show how much each action matters. When a failure mode can lead to more than one loss, its severity is that of the most severe loss, even if that loss is only possible rather than likely. The detection rating describes the checks that exist without the recommended action. Checks that are themselves part of the recommended action do not lower it.

Table 3: Rating scales

Rating	Severity (S)	Occurrence (O)	Detection (D)
9 to 10	Physical harm, including possible harm (L1), legal exposure (L5), or a privacy or security breach (L4)	Almost certain in normal use	Cannot be detected before it reaches the user
7 to 8	Permanent loss or damage of user data (L3), or a serious privacy or security weakness that does not by itself expose data	Frequent	Detected only by chance
5 to 6	The learner learns incorrect content (L2), a goal from the Problem Statement fails for some users, or lasting stress or discouragement (L6)	Occasional	Detected by careful review or targeted testing
3 to 4	User data is lost but can be recovered, the user is clearly inconvenienced, or brief discomfort (L6)	Rare	Usually detected by routine tests
1 to 2	Minor inconvenience with no lasting effect	Very unlikely	Almost certainly detected by automated checks

The risk priority number is $RPN = S \times O \times D$, which ranges from 1 to 1000. A failure mode is **high priority** if $RPN \geq 150$ or $S \geq 9$. The second condition keeps a failure mode that could cause physical harm or legal exposure high priority even when it is unlikely. The ratings are estimates by the team and will be reviewed after the Proof of Concept.

5.4 FMEA Table

The FMEA table is given on the following pages. Table 5, after it, lists the high priority failure modes in order of RPN.

Table 4: FMEA Worksheet

Ref.	Failure Mode	Effects of Failure	Causes of Failure	Detection	Recommended Action	S	O	D	RPN	SR
H1: Lesson Player										
H1.1	Lyric highlighting drifts from the audio by more than 0.5 s.	Learner connects the wrong words to the sounds they hear and may learn the wrong pronunciation (L2). The synchronized playback goal fails.	a. Audio library reports position too coarsely or late. b. App returns from the background, or the user seeks or pauses. c. Timestamps in the map are wrong (see H5.3).	Position logging in the acceptance suite of two songs. Developer debug overlay showing drift.	Drive highlighting from the player's reported position rather than a separate timer, and realign immediately after every seek, pause, resume, and return from the background. If the PoC (risk R1) shows positions are too coarse, switch to a native audio module.	6	7	3	126	IN1, IN2
H1.2	Audio starts unexpectedly or at a sudden high level.	Hearing discomfort or damage with headphones (L1). Audio plays aloud in a public place after headphones are unplugged.	a. Lesson autoplays when opened. b. Playback continues on the phone speaker when headphones disconnect. c. Tracks are mastered at very different loudness.	Manual test on iOS and Android: open lesson, unplug or disconnect headphones during playback.	Never start playback without a learner action. Pause playback when the audio output device is disconnected. Fade audio in when playback starts or resumes. The editor measures the loudness of each track and warns the creator before export when it is far from the target level, so tracks can be normalized.	9	5	5	225	HS1, HS2, HS8, HS9
H1.3	Audio file referenced by a map is missing, corrupt, or in an unsupported format.	Lesson cannot be played, or the app crashes. Loss of queued vocabulary in a crash is covered by H7.1.	a. File not included in the map package. b. Codec not supported on the learner's platform. c. File damaged during transfer.	Check on import that the file exists, decodes, and has a duration consistent with the map.	Reject the lesson on import with a message naming the problem, instead of failing at playback time. Restrict maps to audio formats supported on both iOS and Android.	4	4	3	48	IN11

Continued on next page

Table 4, continued from previous page

Ref.	Failure Mode	Effects of Failure	Causes of Failure	Detection	Recommended Action	S	O	D	RPN	SR
H1.4	Learner watches the screen while walking, cycling, or driving.	Physical injury to the learner or others from distraction (L1).	a. Intended use includes short study moments on the move. b. Following the lesson requires reading highlighted text.	Cannot be reliably detected by the app.	Let a lesson be followed by listening alone: playback continues with the screen locked and needs no taps once started. Show a notice on first launch, advising against using the screen while driving, cycling, or crossing traffic. Avoid interactions under time pressure that demand attention.	10	5	10	500	HS3, HS7
H2: Lyric Display and Language Support										
H2.1	Words in a script that is not Latin and has no spaces between words are segmented incorrectly.	Learner studies something that is not a word or a wrong word boundary and may send it to Anki (L2).	a. Tokenizer dictionary does not cover slang, fragments, or mixed scripts found in lyrics. b. Segmentation differs between editor and app versions.	Comparison against a segmentation checked by the team for the acceptance suite (PoC pass criterion R2).	Compute segmentation in the editor, have the creator verify it line by line, and store it in the song map. The learner app displays the stored segmentation and never segments it again at runtime.	6	8	4	192	IN4
H2.2	Transliteration or reading is wrong (for example, a kanji with several possible readings).	Learner learns the wrong pronunciation (L2).	a. Automatic reading chooses the wrong reading for the context. b. Creator enters a reading incorrectly.	Creator verification step in the editor. Acceptance suite comparison.	Store readings in the song map. Mark automatically generated readings as *unverified* until the creator confirms them, and block export of a map with unverified lines.	6	7	5	210	IN4
H2.3	Lyrics in a supported script do not render correctly (missing glyphs or wrong characters).	Lyrics are unreadable or misleading (L2).	a. Device has no font for the script. b. Text encoding is mishandled when the map is read.	Rendering check of every acceptance suite line on both platforms.	Bundle fonts that cover every script of the supported languages, and read and write all song maps as UTF-8.	5	3	3	45	IN12

Continued on next page

Table 4, continued from previous page

Ref.	Failure Mode	Effects of Failure	Causes of Failure	Detection	Recommended Action	S	O	D	RPN	SR
H2.4	Highlight changes flash rapidly across large areas of the screen.	Discomfort or seizure risk for photosensitive learners (L1).	a. Fast songs with very short words. b. Highlight style changes the background colour of whole lines.	Review against WCAG 2.1 Success Criterion 2.3.1, *Three Flashes or Below Threshold* [2, 3]. Count highlight changes per second in fast test songs.	Use a highlight style that does not flash large areas (for example, an underline or a moving marker). Limit large area changes to three per second, and follow the operating system's reduce motion setting.	9	3	5	135	HS4
H2.5	Active line or word is shown only by colour, or text cannot be enlarged.	Learners with low vision or colour vision deficiency cannot follow the lesson. Strain from small text (L6).	a. Highlight relies on hue alone. b. Fixed font sizes.	Accessibility review with the OS text size setting at its largest and a colour blindness simulator.	Indicate the active line and word with a cue other than colour (weight, underline, marker), and respect the device's text size setting.	5	6	3	90	HS5
H3: Annotation and Lesson Content										
H3.1	A translation or contextual annotation is incorrect.	Learner learns the wrong meaning and reinforces it through Anki reviews (L2).	a. Creator error or misunderstanding of slang or idiom. b. Machine translation copied in without review.	Review by a second team member before publishing. Learner error reports.	Record the author and reviewer of each annotation. The editor does not mark a map as published until every annotation has a reviewer other than its author. Let learners prepare an error report on any line and share it through the phone's share sheet.	6	6	6	216	IN5, AU1, AU2
H3.2	A lesson contains explicit, offensive, or culturally sensitive language without warning.	Learner is upset or uses an offensive term without knowing its register in conversation (L2, L6). Content unsuitable for younger learners.	a. Authentic lyrics often contain profanity, slurs, or sensitive themes. b. Annotations translate a term without describing how offensive it is.	Creator checklist before publishing. Learner error reports.	Each song map carries an explicit content flag. Flagged lessons show a warning before opening and can be hidden through a learner setting. Annotations of vulgar or offensive terms must include a usage note describing their register.	6	7	5	210	HS6

Continued on next page

Table 4, continued from previous page

Ref.	Failure Mode	Effects of Failure	Causes of Failure	Detection	Recommended Action	S	O	D	RPN	SR
H3.3	App design encourages compulsive use or discourages the learner.	Stress, guilt, or loss of motivation (L6).	a. Streaks, loss penalties, or repeated notifications used to drive engagement. b. Learner opens lessons far above their level.	Design review against HS7. Feedback from usability testing (Winter custom document).	Do not use streak loss penalties, countdowns, or unsolicited push notifications. Show each lesson's difficulty level before it is opened.	5	4	7	140	HS7
H4: Song Map Format and Validator										
H4.1	A map passes schema validation but its content is inconsistent.	Desynchronization, missing annotations, or a crash during the lesson (L2, L3).	a. JSON Schema [4] checks structure, not relationships (for example, overlapping or decreasing timestamps, word times outside their line, times past the end of the audio).	Semantic validation in the editor on export and in the app on import. Validator unit tests in CI.	Add semantic rules on top of the schema: lines are in order and do not overlap, word times fall inside their line, all times fall within the audio duration, and every annotated target has both a translation and an annotation (the full list is the SRS definition of a valid song map).	6	5	3	90	IN2
H4.2	Editor and app use different versions of the song map format.	Fields are silently ignored or misread, or the app crashes (L2, L3).	a. Format changes during development. b. Learner has an older app version.	Version check on import.	Each map declares its format version. The app refuses maps with an unsupported major version and says why, instead of opening them partially.	5	5	4	100	IN3
H4.3	A crafted or malformed map is opened.	App or editor freezes or crashes. Script injected through annotation text runs in the web editor (L4).	a. Very large or deeply nested JSON. b. Annotation text containing HTML or script. c. Maps are files that can be shared and modified outside the editor.	Fuzz and boundary tests of the parser. Size and depth checks before parsing.	Reject maps above a size and nesting depth limit before full parsing. Render all map text as plain text in both the app and the editor. Never interpret it as HTML.	9	2	5	90	IM1

Continued on next page

Table 4, continued from previous page

Ref.	Failure Mode	Effects of Failure	Causes of Failure	Detection	Recommended Action	S	O	D	RPN	SR
H4.4	A map references audio or other resources by an external URL.	Learner's device contacts an unknown server, revealing its IP address and usage, or loads unexpected content (L4).	a. Creator links to hosted audio. b. Map is modified by a third party.	Validator rule on all resource paths.	Allow only relative paths inside the map package. Reject absolute and remote URLs.	9	3	3	81	IM2
H4.5	A path in a map package points outside the package (for example, with `../`).	On import, the app reads or writes files outside the lesson folder, which can overwrite its stored data or expose other files (L3, L4).	a. Package crafted by a third party. b. Path checks that reject URLs but not relative paths that climb out of the package.	Fuzz tests of the importer with crafted packages.	Resolve every path against the package folder and reject any path that leaves it, including paths with a `..` segment. Extract each package only into a new folder owned by the app.	9	2	6	108	IM2
H5: Song Map Editor										
H5.1	Creator loses unsaved work.	Lost timing and annotation work. The goal of creating a lesson in 45 minutes fails (L3).	a. Browser tab closed or crashed. b. Computer loses power.	An indicator of unsaved changes in the editor.	Autosave the draft to browser storage at least every 60 seconds and offer to restore it when the editor reopens. Warn before the page closes with unsaved changes.	7	5	3	105	IN6
H5.2	Creator accidentally overwrites or deletes timings or annotations.	Corrupted map that may be exported without the creator noticing (L2, L3).	a. Bulk shift of timestamps applied to the wrong range. b. Accidental delete.	Preview playback before export. Export history (AU2).	Provide undo and redo for all edits, and ask for confirmation before any bulk operation that changes more than one line.	7	5	5	175	IN6, AU2
H5.3	Timestamps recorded in the editor are offset from the audio.	Every lesson made on that computer is out of sync (L2).	a. Browser or Bluetooth audio output latency. b. Creator reaction time when tapping along.	Preview playback in the editor. Acceptance suite timing check.	Provide a latency calibration step whose offset is applied to recorded timestamps, and a preview mode that plays the map exactly as the app will.	6	7	4	168	IN7

Continued on next page

Table 4, continued from previous page

Ref.	Failure Mode	Effects of Failure	Causes of Failure	Detection	Recommended Action	S	O	D	RPN	SR
H5.4	Copyrighted audio or lyrics are used without permission.	Takedown requests or legal liability for the team and creators (L5).	a. Creator uses a commercial recording or lyrics found online. b. Licence is not recorded, so it cannot be checked.	Review of licence metadata before a map is published.	Require source and licence fields for the audio and the lyrics before export, and show the credits in the learner app. Limit the initial release to content that is created by the team, in the public domain, or licensed, as in the Problem Statement.	9	4	5	180	LG1
H6: Anki Handoff										
H6.1	The same vocabulary item is added to Anki more than once.	Cluttered deck and repeated reviews (L3).	a. Learner taps *send* repeatedly. b. Automatic retry after a timeout whose first request actually succeeded.	Duplicate check before sending. Integration test that sends the same item twice.	Before adding a note, check the target deck for an existing note with the same term and context. Disable the send control while a request is in progress.	3	6	3	54	IN9
H6.2	Note fields are filled incorrectly or text is garbled.	Learner reviews a wrong or unreadable card (L2).	a. Fields assigned by position and the note type has a different order. b. Text in a script other than Latin is not encoded as UTF-8.	Check that the note type has the expected fields before sending. Acceptance test of the *Term*, *Translation*, and *Context* fields.	Fill fields by name, not by position. Create the LingoLyrics note type if it is missing and stop with a message if an existing one lacks the required fields. Send all text as UTF-8.	5	4	3	60	IN10
H6.3	The app changes notes, decks, or note types other than its own.	Learner's existing study data or review history is damaged (L3).	a. Programming error in the integration. b. API calls with broader effects than intended.	Code review of every Anki API call. Integration test against a test collection.	The app only adds notes, and only to the LingoLyrics deck. It never edits or deletes existing notes, decks, or note types.	8	2	5	80	AC1
H6.4	Handoff fails silently.	Learner believes the word was saved, and it is lost (L3).	a. Anki or AnkiDroid not installed or not running. b. Permission denied. c. App closed during the request.	Response from Anki checked for every request.	Report success only after Anki confirms the note was created. Keep failed items in a local queue that survives an app restart, and allow retrying them or exporting them to a file.	7	5	6	210	IN8

Continued on next page

Table 4, continued from previous page

Ref.	Failure Mode	Effects of Failure	Causes of Failure	Detection	Recommended Action	S	O	D	RPN	SR
H6.5	AnkiConnect is configured to accept requests from any web page or any computer on the network.	Any website the learner visits, or any device on the same network, can read or change their Anki collection (L3, L4).	a. Setup instructions tell users to allow all origins (`*`) to make the connection work. b. AnkiConnect listens only on `127.0.0.1` by default [7], so reaching it from a phone requires binding it to the network.	Review of setup documentation and default configuration.	If AnkiConnect is chosen (PoC risk R3), allow only the specific origin the product needs, never a wildcard, and require AnkiConnect's API key (`apiKey`) [7]. Warn the learner before any setup step that binds AnkiConnect to an address other than `127.0.0.1`.	8	3	7	168	AC2
H6.6	App requests platform permissions it does not need, or requests them without explanation.	Learner grants access they do not understand, or refuses and loses the feature (L4).	a. Permissions declared and requested at install or first launch.	Review of the app manifest and permission prompts.	Request the AnkiDroid permission [6] only when the learner first sends a word, with a short explanation of why it is needed. Declare no other sensitive permissions.	4	4	3	48	PR2
H7: Local Data Storage										
H7.1	App crashes or is closed while vocabulary is waiting to be sent.	Selected vocabulary is lost (L3).	a. Queue held only in memory. b. Operating system terminates the app in the background.	Restart test with items in the queue.	Store the pending queue on the device as soon as an item is selected, and remove an item only after Anki confirms it.	7	4	5	140	IN8
H7.2	Personal or usage data leaves the device.	Privacy breach, including the learner's vocabulary history and listening habits (L4).	a. Analytics or crash reporting libraries from other vendors. b. Remote resources referenced by maps (H4.4).	Review of dependencies and network traffic during testing.	The initial release sends no data off the device, apart from notes the learner explicitly sends to Anki and error reports the learner chooses to share (AU1). Any analytics or crash reporting added later must stay off unless the learner turns it on.	9	2	6	108	PR1, IM2

Continued on next page

Table 4, continued from previous page

Ref.	Failure Mode	Effects of Failure	Causes of Failure	Detection	Recommended Action	S	O	D	RPN	SR
H7.3	A library from another vendor that the app or editor uses is compromised or has a known vulnerability.	Harmful code runs with full access to the app's or editor's data, or an attacker exploits the flaw through a song map (L3, L4).	a. A dependency, or one of its own dependencies, is taken over and a harmful version is published. b. Versions update without review. c. Known vulnerabilities are not checked.	Dependency review before release (PR1).	Pin exact versions in a committed lock file. Run an automated vulnerability check in CI on every pull request and fail the build on high or critical findings. Have a second team member review every new dependency and every major version change before it is merged.	9	3	7	189	IM3

Table 5: High priority failure modes, in order of RPN

Ref.	Failure mode	S	O	D	RPN	SR
H1.4	Learner watches the screen while moving	10	5	10	500	HS3, HS7
H1.2	Audio starts unexpectedly or loudly	9	5	5	225	HS1, HS2, HS8, HS9
H3.1	Incorrect translation or annotation	6	6	6	216	IN5, AU1, AU2
H6.4	Anki handoff fails silently	7	5	6	210	IN8
H2.2	Incorrect transliteration or reading	6	7	5	210	IN4
H3.2	Explicit content without warning	6	7	5	210	HS6
H2.1	Incorrect segmentation	6	8	4	192	IN4
H7.3	Compromised or vulnerable library	9	3	7	189	IM3
H5.4	Unlicensed audio or lyrics	9	4	5	180	LG1
H5.2	Creator overwrites timings or annotations	7	5	5	175	IN6, AU2
H5.3	Editor timestamps offset from the audio	6	7	4	168	IN7
H6.5	AnkiConnect reachable by any web page or computer	8	3	7	168	AC2
H2.4	Rapid flashing	9	3	5	135	HS4
H7.2	Personal or usage data leaves the device	9	2	6	108	PR1, IM2
H4.5	Map path points outside the package	9	2	6	108	IM2
H4.3	Crafted or malformed map	9	2	5	90	IM1
H4.4	Map refers to external URLs	9	3	3	81	IM2

6 Safety and Security Requirements

The requirements below were discovered through the FMEA. Each requirement lists the FMEA rows it mitigates. All of them are new requirements and have been added to the SRS with the same identifiers, where each one also has a rationale and a fit criterion. HS1 to HS9 are under the safety critical requirements, AC, IN, PR, AU, and IM are under the security requirements, and LG1 is under the legal requirements. In the SRS, the numeric values used here (1 second, three flashes per second, 60 seconds, 0.5 seconds, −16 LUFS, and 2 LU) are written as the symbolic constants `DISCONNECT_PAUSE`, `MAX_FLASHES`, `AUTOSAVE_INT`, `FADE_IN`, `TARGET_LOUDNESS`, and `LOUDNESS_TOL`. The values of these and of `AUDIO_DUR_TOL`, `MAX_MAP_SIZE`, and `MAX_NESTING` are listed in SRS Appendix A, *Symbolic Constants*. The SRS also lists, for each FMEA row, the requirements that mitigate it (SRS Appendix B, *Hazards to Requirements*).

6.1 Health and Safety Requirements

HS1. The learner app shall not start audio playback without an explicit action by the learner. *(H1.2)*

HS2. The learner app shall pause playback within 1 second when the active audio output device is disconnected. *(H1.2)*

HS3. Once a lesson has been started, the learner app shall continue playing it to the end, with no interaction required, while the screen is locked or the app is in the background. The app shall show a notice on first launch advising against using the screen while driving, cycling, or crossing traffic. *(H1.4)*

HS4. Lyric highlighting shall not produce more than three large area flashes in any period of one second [3], and shall follow the operating system's reduce motion setting. *(H2.4)*

HS5. The learner app shall indicate the active line and word with a cue other than colour alone, and shall respect the device's text size setting. *(H2.5)*

HS6. Each song map shall include an explicit content flag. Lessons flagged as explicit shall show a warning before opening and can be hidden through a learner setting. Annotations of vulgar or offensive terms shall include a note describing their register. *(H3.2)*

HS7. The learner app shall not use streak loss penalties, countdown timers, or unsolicited push notifications to encourage use, and shall show each lesson's difficulty level before it is opened. *(H1.4, H3.3)*

HS8. When playback starts or resumes, the learner app shall raise the playback volume from silence to its set level over 0.5 seconds. *(H1.2)*

HS9. The editor shall measure the integrated loudness of each track's audio, as defined in ITU-R BS.1770 [5], and display it to the creator. Before export, it shall warn the creator when the loudness differs from the target of −16 LUFS by more than 2 LU. The measured value shall be stored in the song map. *(H1.2)*

6.2 Access Requirements

AC1. The learner app shall only add notes, and only to the LingoLyrics deck. It shall not edit or delete existing notes, decks, or note types in the learner's Anki collection. *(H6.3)*

AC2. If AnkiConnect is used, the product and its setup instructions shall allow only the specific origin the product requires, never a wildcard origin, and shall require AnkiConnect's API key. The setup instructions shall warn the learner before any step that makes AnkiConnect listen on an address other than `127.0.0.1`. *(H6.5)*

6.3 Integrity Requirements

IN1. The learner app shall drive lyric highlighting from the audio player's reported position and shall realign the highlight after every seek, pause, resume, and return from the background. *(H1.1)*

IN2. The song map validator shall check, in addition to the schema, the conditions of a valid song map given in the SRS (*Formal Specification*, conditions V1 to V8): among them, lines are in order and do not overlap, word times fall within their line, all times fall within the audio duration, segmentation is verified, and every annotated target has both a translation and an annotation. The editor shall run it before export and the learner app shall run it on import. *(H1.1, H4.1)*

IN3. Each song map shall declare its format version. The learner app shall refuse to open a map with an unsupported major version and shall tell the learner why. *(H4.2)*

IN4. Song maps shall store word segmentation and transliteration explicitly, and the learner app shall display the stored values without segmenting again at runtime. The editor shall mark automatically generated values as unverified until the creator confirms them, and shall not export a map containing unverified lines. *(H2.1, H2.2)*

IN5. Each annotation shall record the name of its author and the name of its reviewer. The editor shall not mark a map as published while any annotation has no reviewer or has its author as its reviewer. *(H3.1)*

IN6. The editor shall autosave the current draft at least every 60 seconds, offer to restore it when reopened, warn before closing with unsaved changes, provide undo and redo for all edits, and ask for confirmation before any operation that changes more than one line. *(H5.1, H5.2)*

IN7. The editor shall let the creator set an audio latency offset, either by entering it or by tapping along to a test click track, and shall apply it to recorded timestamps. It shall also provide a preview that plays the map as the learner app will. *(H5.3)*

IN8. The learner app shall report a vocabulary item as sent only after Anki confirms that the note was created. Items that have not been confirmed shall be stored on the device, kept across app restarts, and available to retry or export to a file. *(H6.4, H7.1)*

IN9. Before adding a note, the learner app shall check the LingoLyrics deck for an existing note with the same term and context, and shall disable the send control while a request is in progress. *(H6.1)*

IN10. The learner app shall fill Anki note fields by field name, shall stop with a message if the note type lacks the *Term*, *Translation*, or *Context* field, and shall send all text as UTF-8. *(H6.2)*

IN11. On import, the learner app shall check that the audio file referenced by a map exists, can be decoded, and has a duration within `AUDIO_DUR_TOL` of the duration recorded in the map, and shall reject the lesson with a message if not. *(H1.3)*

IN12. The learner app shall include fonts covering every script used by its supported languages, and all song maps shall be read and written as UTF-8. *(H2.3)*

6.4 Privacy Requirements

PR1. The initial release shall not send any data off the device, other than notes the learner explicitly sends to Anki and error reports the learner chooses to share (AU1). Any analytics or crash reporting added later shall stay off unless the learner turns it on. *(H7.2)*

PR2. The learner app shall request the AnkiDroid permission only when the learner first sends a word, with an explanation of why it is needed, and shall request no other sensitive permissions. *(H6.6)*

6.5 Audit Requirements

AU1. The learner app shall let a learner prepare a report of a suspected error on any lyric line. The report shall contain the map identifier, map version, line, and the learner's comment. The app shall pass the report to the operating system's share sheet, so that the learner decides whether and where to send it (for example, by email or to the project's issue tracker), and shall not send it itself. *(H3.1)*

AU2. Each song map shall record a version number and, for every export, the date and the creator who exported it. *(H3.1, H5.2)*

6.6 Immunity Requirements

IM1. The song map parser in both the editor and the learner app shall reject a map whose size exceeds `MAX_MAP_SIZE` or whose nesting depth exceeds `MAX_NESTING` before parsing it fully, and both shall display all map text as plain text, never as HTML. *(H4.3)*

IM2. Song maps shall refer to audio and other resources only by relative paths that resolve inside the map package. The validator shall reject absolute paths, remote URLs, and any path that leaves the package, such as one containing a `..` segment. The learner app shall extract each imported package only into a new folder that it owns. *(H4.4, H4.5, H7.2)*

IM3. The project shall pin the exact version of every library from other vendors in a committed lock file. CI shall run an automated vulnerability check on every pull request and fail when a high or critical vulnerability is reported. Every new dependency and every major version change shall be reviewed by a team member other than the one who added it before it is merged. *(H7.3)*

6.7 Legal Requirements

LG1. The editor shall require source and licence information for the audio and lyrics before a map can be exported, and the learner app shall show these credits for each lesson. *(H5.4)*

7 Roadmap

The requirements in Section 6 have been written to the scope that the team plans to implement in the capstone timeline. Every one of them will be implemented within that timeline, so every high priority failure mode in Table 5 is mitigated by all of its requirements, not only by some of them:

- **Capstone timeline:** HS1 to HS9, AC1, IN1 to IN12, PR1, PR2, AU1, AU2, IM1 to IM3, and LG1. HS7 and PR1 constrain the design rather than add features, so they are met by not adding engagement mechanics or analytics.
- **Depends on the Proof of Concept:** AC2 applies only if AnkiConnect is chosen for the Anki handoff (PoC risk R3), and will be implemented in the capstone timeline if it is. The detailed behaviour of IN8 to IN10 and PR2 will be confirmed once the handoff method is decided.

Three requirements were narrowed so that they can be met in the capstone timeline. The parts removed are future work, and the risk they would have reduced is accepted in Table 6:

- **HS3** requires that a started lesson can be followed by listening alone, with the screen locked, and the notice on first launch. A separate listening mode designed for use while moving (for example, larger controls or spoken line translations) is future work. Whether it is needed will be decided from usability testing.
- **IN5** enforces in the editor that each annotation names a reviewer other than its author. Because the initial release has no user accounts, the names are entered by the creator and cannot be verified. Verified reviewer identities need accounts and are future work.
- **AU1** prepares the error report in the app and hands it to the operating system's share sheet. The product has no backend to receive reports, so delivering them automatically to creators is future work.

IN7 requires a latency offset entered by the creator or measured by tapping along to a click track. Calibration that measures output latency without the creator's help is not required.

7.1 Residual Risk

Table 6 lists, for each high priority failure mode, the risk that remains once its requirements are implemented and the reason the team accepts it. The remaining risks will be rated again, with the requirements in place, when the hazard analysis is reviewed.

Table 6: Residual risk accepted for each high priority failure mode

Ref.	Residual risk	Reason for accepting it
H1.4	A learner may still look at the screen while walking, cycling, or driving. The app cannot detect this.	The lesson can be followed by listening alone (HS3), and nothing in the app demands attention under time pressure (HS7). The decision to look at the screen is the learner's.
H1.2	The learner may set a harmful volume. Pausing on disconnection depends on the operating system reporting the event. A creator may ignore the loudness warning.	The overall volume is the learner's choice and is covered by the operating system's hearing protection features (Section 5). The fade in (HS8) still prevents a sudden start.
H3.1	A reviewer may miss an error. Reviewer names cannot be verified (IN5). Reports reach creators only if learners send them (AU1).	In the initial release, all lessons are made and reviewed by the team, so the reviewer is known. AU2 lets a corrected map be traced.
H6.4	Queued items are lost if the app is uninstalled or the device's storage fails.	These events are outside the app's control. The learner can export queued items to a file at any time (IN8).
H2.2, H2.1	A creator may confirm a wrong segmentation or reading.	Each value is checked by a person once (IN4). The acceptance suite is compared against a reference checked by the team, and learners can report errors (AU1).
H3.2	A creator may fail to set the explicit content flag or the register note.	The flag is a required field (HS6), the reviewer checks annotations (IN5), and learners can report errors (AU1).

Ref.	Residual risk	Reason for accepting it
H7.3	A new vulnerability may be published after a release, or a harmful version may not yet be known when it is reviewed.	The lock file stops unreviewed versions from entering a build, and the vulnerability check runs on every pull request, so a newly reported flaw is caught at the next change (IM3).
H5.4	A creator may enter false source or licence information. The product cannot verify it.	The initial release distributes only content that the team has checked is created by the team, in the public domain, or licensed (BR3 in the SRS).
H5.2	A bulk operation that the creator confirmed may still be wrong.	Undo, the confirmation, and the preview make the error visible and reversible (IN6). AU2 records each export.
H5.3	Output latency that changes during a session (for example, with Bluetooth) is not corrected by a single offset.	The preview (IN7) shows the error before export, and the creator can calibrate again.
H6.5	The learner may change the AnkiConnect configuration themselves, for example by allowing all origins.	The product's instructions never ask for this (AC2), and the API key still protects the collection. Applies only if AnkiConnect is chosen.
H2.4	Flash limits are measured on the acceptance suite. A faster song may still exceed them.	The highlight style does not change large areas (HS4), so a fast song produces small changes rather than large flashes.
H7.2	An updated dependency may add network traffic. A shared error report contains what the learner writes.	The dependency review and network capture of the PR1 fit criterion are repeated before each release. The learner chooses whether to share a report and where to send it (AU1).
H4.5	A defect in the operating system's file handling may still let a path escape the folder.	Paths are checked after they are resolved, and each package is extracted into a new empty folder (IM2), so an escape could only reach the app's own storage.
H4.3	A defect in the JSON parser library may still be exploitable.	Size and depth limits are applied before parsing, and map text is never interpreted as HTML (IM1), which limits what a defect can do.
H4.4	A defect in the validator may accept a remote URL.	The validator rule is tested with maps containing absolute paths and URLs (IM2), and the app has no other code that fetches resources from the network.

The hazard analysis will be reviewed after the Proof of Concept and again before the final demonstration to confirm which hazards have been mitigated and which remain open.

References

[1] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, MA: MIT Press, 2011. doi:10.7551/mitpress/8179.001.0001.

[2] World Wide Web Consortium, *Web Content Accessibility Guidelines (WCAG) 2.1*, W3C Recommendation. https://www.w3.org/TR/WCAG21/

[3] World Wide Web Consortium, *Understanding Success Criterion 2.3.1: Three Flashes or Below Threshold*. https://www.w3.org/WAI/WCAG21/Understanding/three-flashes-or-below-threshold.html

[4] JSON Schema, *Draft 2020-12*. https://json-schema.org/draft/2020-12

[5] International Telecommunication Union, *Recommendation ITU-R BS.1770: Algorithms to Measure Audio Programme Loudness and True-Peak Audio Level*. https://www.itu.int/rec/R-REC-BS.1770

[6] AnkiDroid, *AnkiDroid API* (project wiki). https://github.com/ankidroid/Anki-Android/wiki/AnkiDroid-API

[7] A. Yatskov (FooSoft), *AnkiConnect* (Anki extension documentation). https://git.sr.ht/~foosoft/anki-connect

All web sources were accessed on October 18, 2026.

8 Appendix: Generative AI Use Disclosure

8.1 AI Tools Used

Table 7: AI tools used for this document

Tool	Version	Notes
Claude Code (Anthropic)	2.1.158	Command line coding assistant, used inside VS Code with access to the repository
Claude Opus (Anthropic)	5.5 (`claude-opus-5-5`)	The model used by Claude Code for every task below

No other AI tools were used. The FMEA rows, ratings, and requirements were first drafted by the team. AI was used to review and revise them, never to produce the analysis without a team member checking the result.

8.2 How and Where Used

Claude was given the Problem Statement and Goals document, the Development Plan, the SRS, the course template and Hazard Analysis checklist, the grading rubric, and three example hazard analyses from previous capstone projects. Table 8 lists each use, the sections it affected, and the date.

Table 8: Where AI was used in this document

Date	Task	Sections affected
Oct 18, 2026	Review the first draft for grammar, and compare it with previous student work and the checklist to find missing sections or details	Whole document
Oct 18, 2026	Compare the document with the SRS so that no relevant requirement was missed	Section 6
Oct 18, 2026	Address review comments: check every rating against the scales, add HS8 and HS9, narrow HS3, IN5, IN7, and AU1 to the capstone scope, write the residual risk table, and add references	Sections 5, 6, and 7
Oct 19, 2026	Check the document against the grading rubric, suggest the document overview, context figure, interface table, method description, CA3, H4.5, H7.3, and IM3, and reword the text to remove hyphenated words and semicolons	Sections 2 to 7
Oct 19, 2026	Draft the reflection answers from the commit history and the review comments, for the team to edit	Reflection appendix

A typical prompt was: “Review the HA against this rubric and make the changes needed for full marks.” Claude’s suggestions were made as edits in a branch and reviewed as a diff before they were committed, so every change can be seen in the repository history.

8.3 Verification of Content Generated by AI

Content written or changed with Claude was checked as follows:

- **Ratings:** each severity, occurrence, and detection rating in Table 4 was compared with the definitions in Table 3, using the rules stated with the table (the most severe possible loss sets the severity, and checks that are part of the recommended action do not lower the detection rating). Each RPN was recalculated, and Table 5 was rebuilt from the recalculated values.

- **External claims:** statements about Anki, AnkiDroid, AnkiConnect, WCAG, JSON Schema, and ITU-R BS.1770 were checked against the primary sources in the References section. This check found that AnkiConnect supports an API key and listens only on `127.0.0.1` by default, and H6.5 and AC2 were revised to match.
- **Consistency with the SRS:** a script checked that every RPN equals $S \times O \times D$, that Table 5 and Table 6 contain exactly the failure modes that meet the high priority rule, that every requirement in this document exists in the SRS and traces to the same FMEA rows, and that the SRS table *Hazards to Requirements* matches the SR column of Table 4. The wording of each requirement was also compared. It matches, except that the SRS writes numeric values as symbolic constants. The check was run again after H4.5, H7.3, and IM3 were added in the October 19 revision.
- **New failure modes:** H4.5 and H7.3 were accepted only after a team member confirmed that each describes a real weakness of the planned design (song maps are shared as packages of files, and both the app and the editor depend on many npm packages) and that no existing row already covered it.
- **Build check:** the document was compiled after every revision, and the table of contents, lists of tables and figures, and all cross references were checked in the PDF.
- **Reflection:** the drafted answers were read by each team member, and anything that did not match their experience was rewritten in their own words.

8.4 Limitations

The ratings are the team's judgement, and AI was used to check them for consistency, not to decide them. AI cannot confirm how often a failure will happen in practice, so the occurrence ratings will be checked again with data from the Proof of Concept and usability testing.

9 Appendix: Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. **What went well while writing this deliverable?**
   Writing the hazard analysis alongside the SRS worked well. Each recommended action became a requirement with a fit criterion in the SRS straight away, so the two documents grew together instead of being matched up at the end. Going through the system one component at a time also kept the work easy to divide, since each component already has an owner in the Development Plan. The rating scales were the other thing that went well. Once the scales were tied to the losses L1 to L6, most disagreements about a rating could be settled by pointing to a row of Table 3 rather than arguing from opinion.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**
   The biggest pain point was rating consistently. In the first draft, similar failure modes had quite different scores, and some detection ratings were lowered by checks that were really part of the recommended action. The review comments pointed this out. We resolved it by writing down two rules next to the scales (the worst possible loss sets the severity, and the recommended action does not count towards detection) and then rating every row again from scratch.

   The second pain point was scope. Some of our first recommended actions, such as verified reviewer identities and automatic delivery of error reports, would need user accounts and a backend that the initial release does not have. We could not simply drop them, because the risks are real. We resolved this by narrowing HS3, IN5, IN7, and AU1 to what we can build in the capstone timeline and recording the risk we accept in Table 6, so the gap is visible instead of hidden.

   Keeping the hazard analysis and the SRS in step was also tedious, since a change to one requirement meant editing two documents and a traceability table. A small script that compares the requirement identifiers and FMEA references in both documents made this much less error prone.

3. **Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about?**
   Before this deliverable we had already identified the three Proof of Concept risks in the Development Plan: lyric sync on a phone (R1), which became H1.1, segmenting lyrics (R2), which became H2.1 and H2.2, and sending words to Anki from a phone (R3), which became the H6 rows. We had also discussed copyright from the start, since the Problem Statement limits content to material that is created by the team, in the public domain, or licensed (H5.4). At that point we thought of all of these as technical or project risks, not as harm to users.

   Most of the other rows were new. They came about in three ways.

   - *Thinking about how the app is really used.* Picturing a learner with headphones on a bus or walking to class led to sudden loud audio (H1.2), distraction while moving (H1.4), and audio switching to the speaker in public when headphones disconnect.
   - *The checklist's prompt to think broadly about safety.* We had not seen a language app as something that could harm wellbeing, but the checklist made us look at photosensitivity (H2.4), explicit lyrics without warning (H3.2), and streaks and notifications that create guilt (H3.3). Authentic songs

often contain profanity, and a learner who repeats a slur without knowing its register is a real harm we had missed.

- *Treating song maps as untrusted input.* Once we accepted that a map is a file anyone can edit and share, security hazards followed: crafted maps (H4.3), external URLs (H4.4), and paths that escape the package (H4.5). Reading the AnkiConnect documentation to check our claims showed that it listens only on `127.0.0.1` by default, which made us realize that our own setup instructions could be the thing that opens it to the network (H6.5). Checking the boundary for every input also reminded us that the libraries we depend on are an input as well (H7.3).

The main lesson for us is that most of the hazards were not about the software failing. They exist when the software works exactly as designed, which is why we would not have found them by thinking only about bugs.

4. **Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?**
   Security and privacy risk: software can expose or misuse personal data, or let an attacker run code or change data. For LingoLyrics, a shared song map is untrusted input and the app writes into the learner's Anki collection, so a flaw could damage data the learner has built up over years. These risks matter because users usually cannot see them and the harm is often permanent.

   Psychological and wellbeing risk: engagement mechanics such as streaks and notifications can create guilt, stress, or compulsive use, and offensive content without warning can upset users. These risks matter because the people most affected, such as young or vulnerable users, are least able to recognize them, and they grow with the number of users.

   Legal and content accuracy risk: distributing unlicensed content or teaching incorrect information can cause legal liability or lasting harm to users who trust the product. For a learning tool, incorrect content is the main way it can fail its users even when the software works perfectly.